STAT 185 - Problem Set #3

Matt Krasnow

Assignment Details and Instructions

This assignment covers Singular Value Decomposition and Nonnegative Matrix Factorization through a mixture of theory and coding using the packages NMF and irlba. You will need the file *pset3.Rdata* available on Canvas. The commands for loading the data assume this file, *pset3.Rmd*, and the Rdata file are in the same folder. Please compile the document into a single pdf and submit your solutions to Canvas by Friday October 10th at 11:59 PM.

The coding portions of this assignment are intensive and may several hours to complete when all run together depending on the computer that you use. Please keep this in mind with regards to due dates. To save time, caching has been enabled, but will only occur when you knit, not when you run chunks as you work on the assignment.

Problem 1

For a matrix $\mathbf{X} \in \mathbb{R}^{N \times d}$, we know from linear algebra that $\text{rank}(\mathbf{X}) \leq \min\{N, d\}$. For a matrix $\mathbf{X} \in \mathbb{R}_+^{N \times d}$ its nonnegative rank is the smallest k such that $\mathbf{X}$ has an exact nonnegative matrix factorization $\mathbf{X} = \mathbf{WH}$ for $\mathbf{W} \in \mathbb{R}_+^{N \times k}$ and $\mathbf{H} \in \mathbb{R}_+^{k \times d}$.

a) Show that the nonnegative matrix rank is also bounded above by the minimum of N and d.

Since $\mathbf{X} \in \mathbb{R}_+^{N \times d}$ is nonnegative, we can always construct an **exact nonnegative factorization** with inner dimension $k = \min\{N, d\}$.
- If $d \leq N$: Set $\mathbf{W} = \mathbf{X} \in \mathbb{R}_+^{N \times d}$ and $\mathbf{H} = \mathbf{I}_d \in \mathbb{R}_+^{d \times d}$. Then $\mathbf{X} = \mathbf{WH}$.
- If $N < d$: Set $\mathbf{W} = \mathbf{I}_N \in \mathbb{R}_+^{N \times N}$ and $\mathbf{H} = \mathbf{X} \in \mathbb{R}_+^{N \times d}$. Then $\mathbf{X} = \mathbf{WH}$.
Thus, there exists an exact nonnegative factorization with $k = \min\{N, d\}$, implying that

$$\text{rank}_+(\mathbf{X}) \leq \min\{N, d\}.$$

b) Show that the rank of $\mathbf{X} \in \mathbb{R}_+^{N \times d}$ is less than or equal to its nonnegative matrix rank.

For any factorization $\mathbf{X} = \mathbf{WH}$ (nonnegative or not) with inner dimension k, we know from basic rank properties that

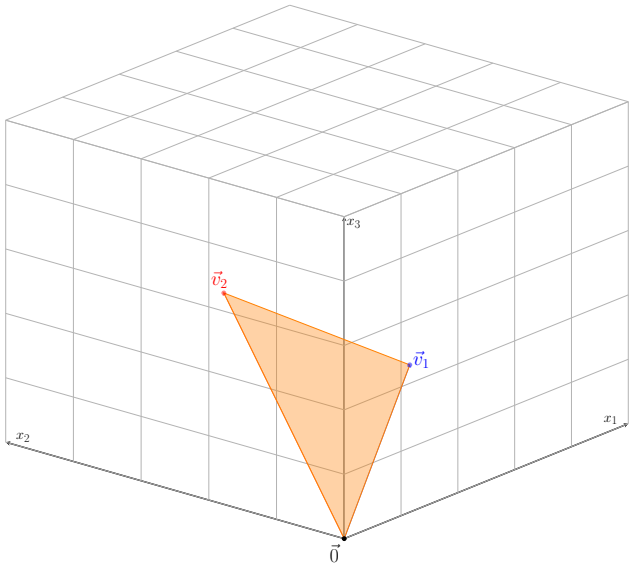
$$\text{rank}(\mathbf{X}) = \text{rank}(\mathbf{WH}) \leq \min\{\text{rank}(\mathbf{W}), \text{rank}(\mathbf{H})\} \leq k.$$

If $\mathbf{W}, \mathbf{H}$ are **nonnegative** and form an exact factorization of $\mathbf{X}$, then the smallest such k satisfying this is by definition $\text{rank}_+(\mathbf{X})$. Therefore,

$$\text{rank}(\mathbf{X}) \leq \text{rank}_+(\mathbf{X}).$$

Problem 2

The convex hull of vectors $\vec{0}$, $\vec{v_1}$, and $\vec{v_2}$ is shaded orange in the figure below.



Note, the volume (in a three dimensional sense) of the orange region is 0, but its area (volume in a two dimensional sense) is positive.

a) Show that the area of the orange region is given by the following formula, $\frac{1}{2}\sqrt{\det(\mathbf{V}^T\mathbf{V})}$ where $\mathbf{V}$ is the matrix with columns $\vec{v}_1$ and $\vec{v}_2$.

The determinant tells us the area of the full parallelogram formed by the two column vectors of $\mathbf{V}$. Since the convex hull is the **triangle** formed by $\{\vec{0}, \vec{v_1}, \vec{v_2}\}$, its area is half of the parallelogram's area.
We know from differential geometry that the **metric tensor** is given by

$$\mathbf{g} = \mathbf{V}^\top \mathbf{V}.$$

The square root of the determinant of $\mathbf{g}$ gives the total area of the parallelogram spanned by $\vec{v_1}$ and $\vec{v_2}$:

$$A_{\text{parallelogram}} = \sqrt{\det(\mathbf{V}^\top \mathbf{V})}.$$

Therefore, the area of the orange triangle (the convex hull) is

$$A_{\text{triangle}} = \frac{1}{2}\sqrt{\det(\mathbf{V}^\top \mathbf{V})}.$$

b) Based on the formula from (a) and the picture above, how would we minimize the area of the convex hull?

Since the formula depends on the inner products between the columns of $\mathbf{V}$, specifically through $\mathbf{V}^\top\mathbf{V}$, we can minimize the area by **increasing** the dot product between $\vec{v_1}$ and $\vec{v_2}$. Increasing the dot product means making the vectors more **collinear** (closer together).
When two vectors are collinear, the angle θ between them approaches 0 or π, and the sine term in the area formula,

$$A = \frac{1}{2}\|\vec{v_1}\|\,\|\vec{v_2}\|\sin(\theta),$$

approaches zero. Thus, as $\vec{v_1}$ and $\vec{v_2}$ become more aligned, $\sin(\theta) \to 0$, and the area of the convex hull approaches zero.
So, to minimize the area, we should make $\vec{v_1}$ and $\vec{v_2}$ **more collinear**, pointing in nearly the same direction.

c) More generally, if we consider k linearly independent vectors $\vec{v}_1, \ldots, \vec{v}_k \in \mathbb{R}^d$, the convex hull of the $k+1$ vectors $\vec{0}, \vec{v_1}, \ldots, \vec{v}_k$ will have positive k dimensional volume. (We can always restrict our focus to the k-dimensional span of these vector and pretend we're working in $\mathbb{R}^k$.)In this case, the k-dimensional volume of the convex hull is

$$\frac{1}{k!}\sqrt{\det(\mathbf{V}^T\mathbf{V})}$$

where again, $\mathbf{V} \in \mathbb{R}^{d\times k}$ has columns $\vec{v}_1, \ldots, \vec{v}_k$. Consider the vectors $\vec{v}_0 = (0,0,0,0)^T$, $\vec{v}_1 = (1,2,0,0)^T$, $\vec{v}_2 = (0,1,1,0)^T$ and $\vec{v}_3 = (0,0,1,0)^T$. We can view these vectors as points in the $w{-}x{-}y{-}z$ plane, specifically points which live in the $z = 0$ subspace. What shape does the convex hull of $\{\vec{v}_j\}_{j=0}^3$ form in that subspace? Compute the volume of the convex hull using basic geometry and verify the provided formula holds.

Since this is a four-pointed shape in a three-dimensional subspace, it should be a tetrahedron (a 3-simplex) since the vectors are linearly independent. You can simply take the determinant of the corresponding matrix to find the volume:

$$\mathbf{V} = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix}.$$

The determinant of $\mathbf{V}$ is

$$\det(\mathbf{V}) = 1(1 \cdot 1 - 0 \cdot 1) - 0(2 \cdot 1 - 0 \cdot 0) + 0(2 \cdot 1 - 0 \cdot 0) = 1.$$

This tells us the volume of the **parallelepiped** is 1, so the **tetrahedron's** volume (since it's a simplex with one vertex at the origin) is

$$V_{\text{tetrahedron}} = \frac{1}{3!} \det(\mathbf{V}) = \frac{1}{6}.$$

Now, checking with the general formula:

$$V = \frac{1}{k!} \sqrt{\det(\mathbf{V}^\top \mathbf{V})}.$$

Compute $\mathbf{V}^\top \mathbf{V}$:

$$\mathbf{V}^\top \mathbf{V} = \begin{bmatrix} 5 & 2 & 0 \\ 2 & 2 & 1 \\ 0 & 1 & 1 \end{bmatrix}.$$

Then,

$$\det(\mathbf{V}^\top \mathbf{V}) = 1.$$

Thus, we confirm that

$$V = \frac{1}{3!} \sqrt{1} = \frac{1}{6},$$

matching the geometric result.

d) Based on the preceding analysis, suppose we adjust the NMF loss function to be

$$\min_{\mathbf{W} \in \mathbb{R}^{d \times k}, \mathbf{H} \in \mathbb{R}^{k \times d}, \mathbf{W}\vec{1} = \vec{1}} \|\mathbf{X} - \mathbf{WH}\|_F^2 + \lambda \sqrt{\det(\mathbf{HH^T})}.$$

What effect does this have on the rows of $\mathbf{H}$ for larger values of λ? How might this improve interpretability of the latent features found by the algorithm? (Hint: the Ang and Gillis paper introducing this penalty is titled "Volume-regularized NMFs".)

For larger values of λ, the regularization term

$$\lambda\sqrt{\det(\mathbf{H}\mathbf{H}^\top)}$$

penalizes the **volume** of the convex hull spanned by the rows of $\mathbf{H}$. Minimizing this term encourages those rows to become more **collinear**, meaning they point in similar directions and occupy a smaller region in latent space.
This effectively makes the rows of $\mathbf{H}$ less spread out — they align more closely and share more variance. The result is that the learned components (latent features) become more distinct and interpretable, since the model avoids redundant or overlapping directions and instead identifies the most representative patterns in the data.
As λ increases, the algorithm increasingly favors smaller, tighter volumes (more collinear latent features), leading to more interpretable results.

Problem 3

The Frobenius norm or KL divergence loss functions can be justified through different assumptions about the data generating process. We'll investigate this idea in more depth in this problem. Suppose we assume

$$\vec{x}_i^T = w_{i1}\vec{h}_1^T + \dots w_{ik}\vec{h}_k^T + \vec{\epsilon}_i$$

where $\vec{\epsilon}_1, \dots, \vec{\epsilon}_N$ are iid $\mathcal{N}(\vec{0}, \sigma^2\mathbf{I})$ random noise. In this case, we are assuming that each row of the associated data matrix $\mathbf{X}$ is a linear combination of the same latent vectors $\vec{h}_1^T, \dots, \vec{h}_k^T$ corrupted by additive, isotropic Gaussian noise. We could express this more compactly in the form

$$\mathbf{X} = \mathbf{W}\mathbf{H} + \mathbf{E}$$

where $\mathbf{E}$ has iid $N(0, \sigma^2)$ entries.

a) Assuming independence of the rows of $\mathbf{X}$ given $\mathbf{W}$ and $\mathbf{H}$, compute the likelihood of $\mathbf{W}$, $\mathbf{H}$ and σ^2 given data $\mathbf{X}$. \begin{tcolorbox}

This is equivalent to

$$P(x^{(i)} \mid W, H, \sigma^2) = \mathcal{N}(x^{(i)} \mid W_i H, \sigma^2 I),$$

where $\Sigma = \sigma^2 I$.

So the likelihood for all rows of X is

$$P(X \mid W, H, \sigma^2) = (2\pi\sigma^2)^{-\frac{Nd}{2}} \exp\left(-\frac{1}{2\sigma^2}\sum_i (x^{(i)} - W_i H)^T (x^{(i)} - W_i H)\right).$$

\end{tcolorbox}

b) Show that finding the maximum likelihood estimate of $\mathbf{W}$ and $\mathbf{H}$ is equivalent to minimizing $\|\mathbf{X}-\mathbf{W}\mathbf{H}\|_F^2$.

$$\arg\max_{W,H} P(X \mid W, H, \sigma^2) = \arg\max_{W,H} \exp\left(-\frac{1}{2\sigma^2}\sum_i (x^{(i)} - W_i H)^T (x^{(i)} - W_i H)\right).$$

Taking the log,

$$\arg\max_{W,H} \log P(X \mid W, H, \sigma^2) = \arg\min_{W,H} \sum_i (x^{(i)} - W_i H)^T (x^{(i)} - W_i H).$$

This is equivalent to minimizing

$$\|X - WH\|_F^2,$$

which gives the maximum likelihood estimate for W and H.

Problem 4

Suppose we want to combine the Itakura-Saito divergence (discussed in the NMF section) with sparsity inducing ℓ_1 penalties for the entries of $\mathbf{W}$ and $\mathbf{H}$. One option is the following NMF problem

$$\min_{\mathbf{W}\in\mathbb{R}_+^{d\times k},\,\mathbf{H}\in\mathbb{R}_+^{k\times d}} \quad \sum_{i=1}^{N}\sum_{j=1}^{d}\left(\frac{\mathbf{X}_{ij}}{(\mathbf{WH})_{ij}} - \log\frac{\mathbf{X}_{ij}}{(\mathbf{WH})_{ij}}\right) + \lambda\sum_{ij}|\mathbf{W}_{ij}| + \lambda\sum_{ij}|\mathbf{H}_{ij}|.$$

a) Compute the gradient of the loss w.r.t. $\mathbf{W}$.

b) Compute the gradient of the loss w.r.t. $\mathbf{H}$.

(a)(b)
$\frac{\partial \mathcal{L}_{IS}}{\partial W} = (-X \circ Y^{-2} + Y^{-1})H^T$
$\frac{\partial \mathcal{L}_{IS}}{\partial H} = W^T(-X \circ Y^{-2} + Y^{-1})$
Add back the sparsity terms
We know $\frac{\partial \|W\|_1}{\partial W_{ij}} = \begin{cases} 1, & W_{ij} > 0 \\ [-1,1], & W_{ij} = 0 \end{cases}$ same for H
$\frac{\partial \mathcal{L}_{IS}}{\partial W} = (-X \circ Y^{-2} + Y^{-1})H^T + \lambda\partial\|W\|$
$\frac{\partial \mathcal{L}_{IS}}{\partial H} = W^T(-X \circ Y^{-2} + Y^{-1}) + \lambda\partial\|H\|$

c) Choose appropriate step sizes to derive multiplicative update rules for $\mathbf{W}$ and $\mathbf{H}$. You may ignore the constraint $\mathbf{W}\vec{1} = \vec{1}$. In practice, this constraint can be implement by rescaling the rows of $\mathbf{W}$ and subsequently the columns of $\mathbf{H}$ after each round of multiplicative updates.

How to make
$W \leftarrow W - \eta\frac{\partial \mathcal{L}_{IS}}{\partial W}$
$H \leftarrow H - \eta\frac{\partial \mathcal{L}_{IS}}{\partial H}$
to a multiplicative update.
So
$W \leftarrow W - \eta[(-X \circ Y^{-2} + Y^{-1})H^T]$
$W_{ij}^{(t+1)} = W_{ij}^{(t)} - \eta_{ij}[(Y^{-1}H^T)_{ij} - (X \circ Y^{-2})H_{ij}^T]$
$= W_{ij}^{(t)} + \eta_{ij}(X \circ Y^{-2})H_{ij}^T - \eta_{ij}(Y^{-1}H^T)_{ij}$
Allows positivity
Need a way to cancel the negative
$\eta_{ij} = \frac{W_{ij}^{(t)}}{(Y^{-1}H^T)_{ij}}$
So
$W_{ij}^{(t+1)} = \frac{W_{ij}^{(t)}((X\circ Y^{-2})H^T)_{ij}}{(Y^{-1}H^T)_{ij}}$
Thus we have a multiplicative update!
We do the same for H and get
$H_{ij}^{(t+1)} = \frac{H_{ij}^{(t)}(W^T(X\circ Y^{-2}))_{ij}}{(W^TY^{-1})_{ij}}$
Now that the math is done, let's add back the λ and go back to matrix-wise computation.

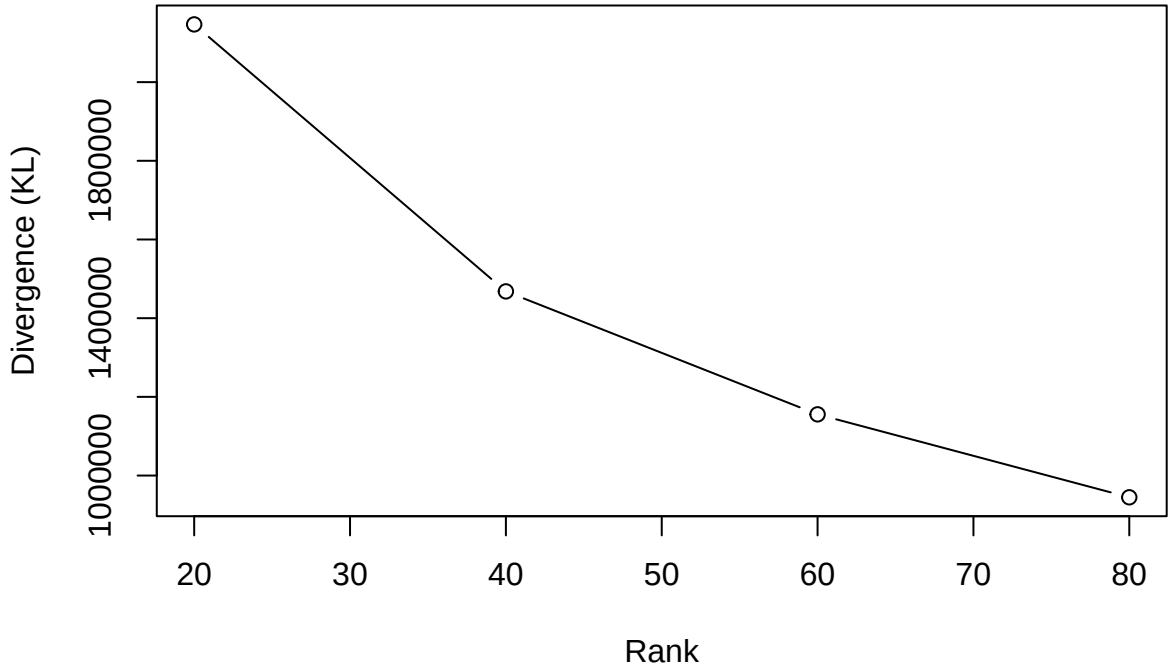
$$\boxed{W \leftarrow W \circ \frac{(X \circ Y^{-2})H^T}{(Y^{-1})H^T + \lambda}, \qquad H \leftarrow H \circ \frac{W^T(X \circ Y^{-2})}{W^T(Y^{-1}) + \lambda}}$$

Problem 5

Let's revisit the Olivetti faces dataset which contains 10 images of 40 different people. Each image is a 64×64 pixellated image. These images have been vectorized and saved in the data matrix *faces* just as in problem set 2, and the function *plot_image* is included to allow you to visualize vectors as images.

a) Compute NMFs of the faces datasets for ranks $20, 40, 60$, and 80 under Divergence loss using only two runs per rank to save computation time. Plot the Divergence error as as function of rank. Is there a clear choice for an appropriate rank? Explain. (Note: fitting the 8 NMF models will likely take one to two hours depending on computer. Plan accordingly.)

```
set.seed(185)
ranks <- c(20, 40, 60, 80)
divergences <- numeric(length(ranks))
fits <- vector("list", length(ranks))
for (i in seq_along(ranks)) {
  fits[[i]] <- nmf(faces, ranks[i], method = "brunet", nrun = 2, .opt = "t")
  Xhat <- fitted(fits[[i]])
  X <- faces
  eps <- .Machine$double.eps
  divergences[i] <- sum(X * log(pmax(X, eps) / pmax(Xhat, eps)) - X + Xhat)
}
plot(ranks, divergences, type = "b", xlab = "Rank", ylab = "Divergence (KL)")
```




```
opt_index <- which.min(divergences)
opt_rank <- ranks[opt_index]
best_fit <- fits[[opt_index]]
opt_rank
```

```
## [1] 80
```

fit NMF with KL loss for ranks 20, 40, 60, and 80. The divergence drops steadily as rank increases, with the lowest value at rank 80. There's no sharp elbow in the curve—just diminishing returns. The improvement from 60 → 80 is smaller than from 20 → 40. If you choose strictly by the loss, use k = 80. If one prefers a simpler model with nearly the same error, k = 60 is a reasonable, more compact choice.

b) Plot all feature images (rows of $\mathbf{H}$ rendered as images) from the optimal rank identified in part (a) using the command *plot_image*.

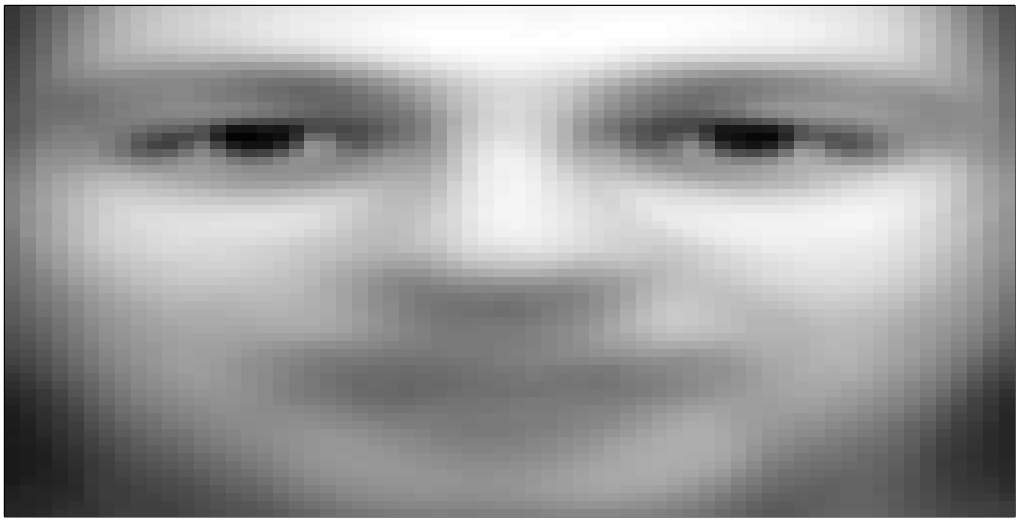
```r
H <- coef(best_fit)
k <- nrow(H)
par(mfrow = c(ceiling(k / 10), 10), mar = c(0, 0, 1, 0))
for (i in 1:k) plot_image(H[i, ])
```



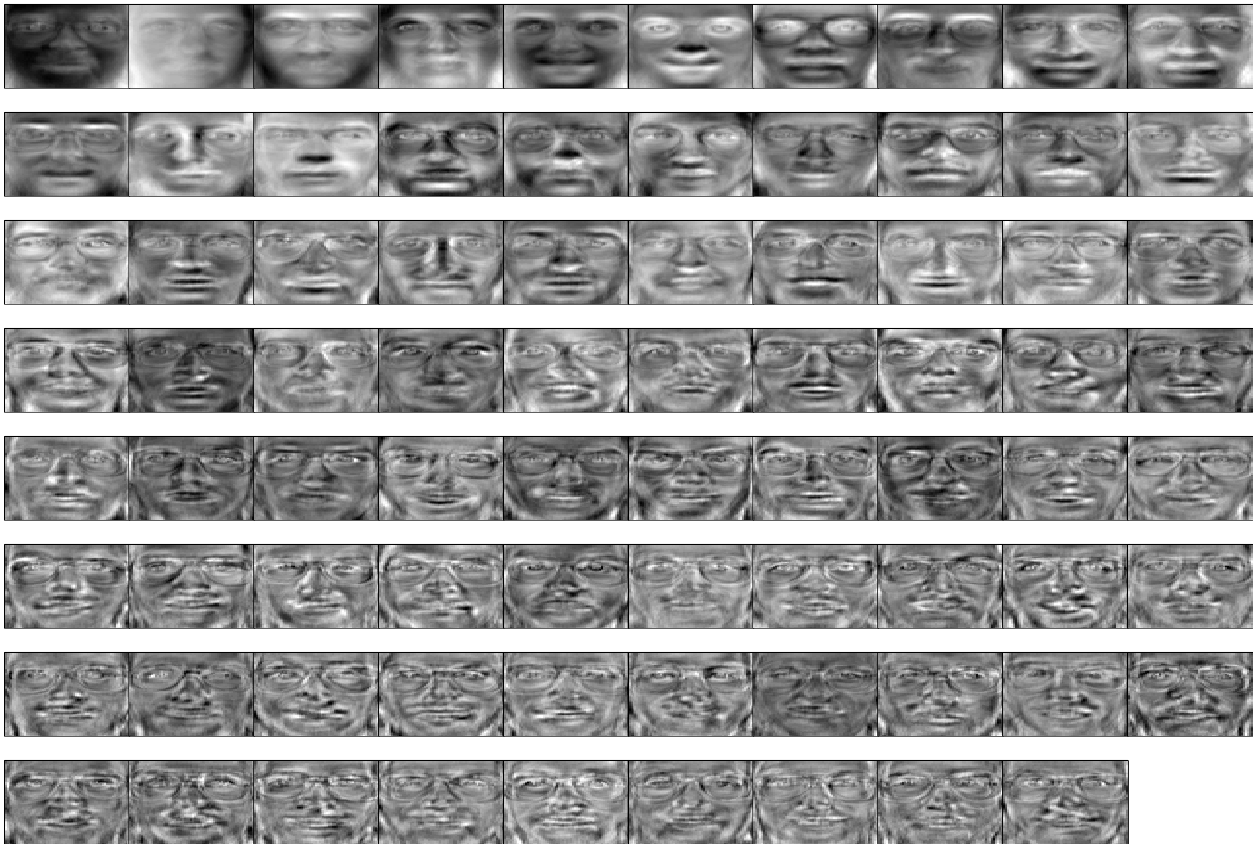
The rank-80 basis images (rows of H) look like parts of faces rather than whole faces. You see localized structure: eyes and eyebrows, mouth lines, cheeks, nose bridges, glasses, and lighting blobs. KL-NMF learns nonnegative, additive components, so each face builds up from a small set of these parts. That makes the features easy to read: a component "means" a region or attribute, not a positive–negative swing across the whole image.

c) For comparison, run PCA on the faces data again. Display the mean image and first $k-1$ loadings as images, where k is the rank you identified. Likewise, run SVD on the faces data and display the first k right singular vectors rendered as images. Which set of features (NMF, SVD, or PCA) look more face-like?

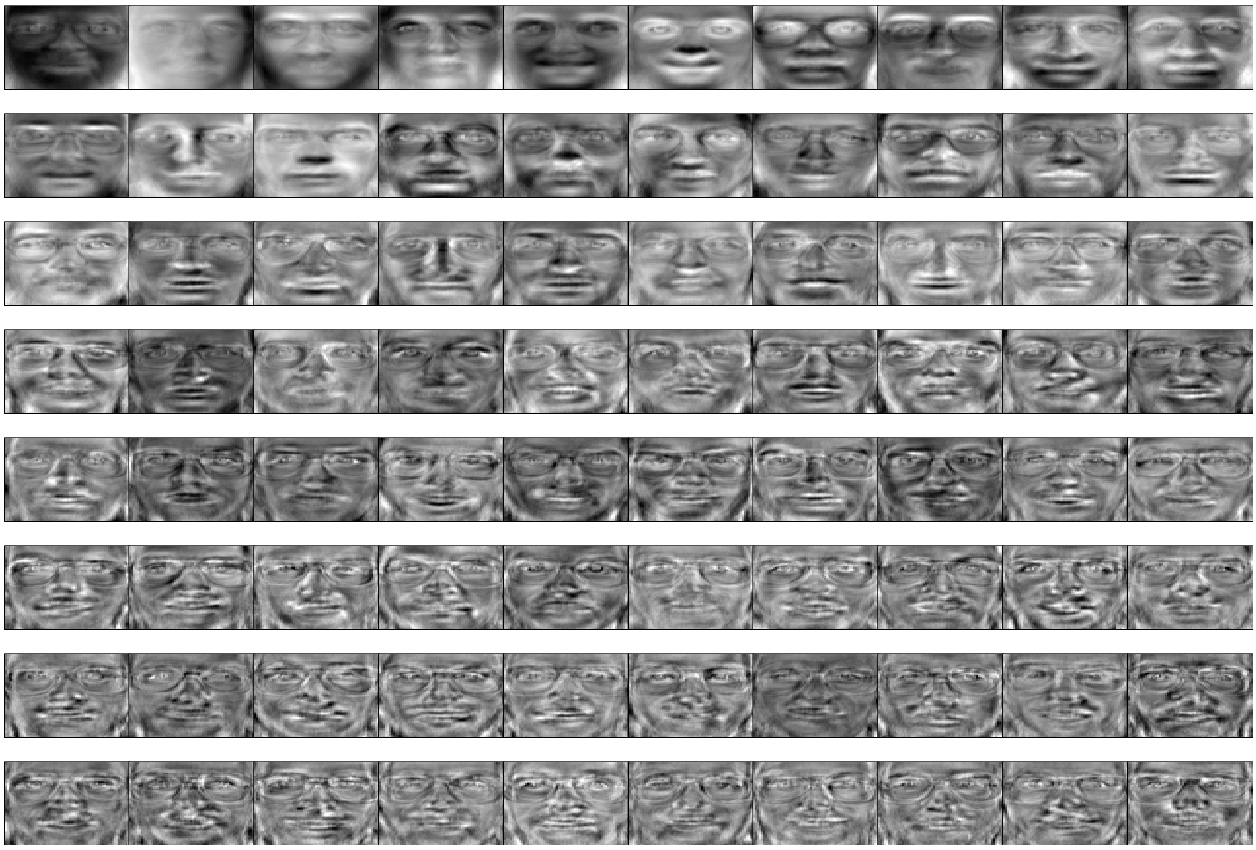
```r
k <- opt_rank
mu <- colMeans(faces)
plot_image(mu)
```



```
pc <- prcomp(faces, center = TRUE, scale. = FALSE)
par(mfrow = c(ceiling((k - 1) / 10), 10), mar = c(0, 0, 1, 0))
for (i in 1:(k - 1)) plot_image(pc$rotation[, i])
Xc <- scale(faces, center = TRUE, scale = FALSE)
sv <- svd(Xc)
par(mfrow = c(ceiling(k / 10), 10), mar = c(0, 0, 1, 0))
```



```
for (i in 1:k) plot_image(sv$v[, i])
```



PCA shows a smooth mean "average face," and the first loadings ("eigenfaces") capture global variation such as illumination, pose, and broad shading. The SVD right singular vectors of the centered data look similar for the same reason. These features are holistic and often mix positive and negative lobes across the image, which is harder to interpret directly.
By contrast, the NMF features are sparser and more localized, so they read as face parts you can name. For interpretability and "face-likeness" of individual components, NMF wins. For energy capture with few components and orthogonality, PCA/SVD are stronger. Here, if your goal is human-readable features, prefer the NMF basis from part (b).

UL in Action: A Simple Recommender System

Problem 6

A subset of reviews from the movielens data in pset #1 has been converted to a matrix of ratings contained in the variable *rating*. In that matrix, each row corresponds to a different reviewer. Each column corresponds to a different movie and are labeled accordingly. Here are the first few entries of that matrix.

```
knitr::kable(rating.matrix[1:5,1:3], "simple")
```

Toy Story (1995)	Sabrina (1995)	Leaving Las Vegas (1995)
1	3	3
NA	NA	NA
NA	NA	NA
NA	3	3
NA	NA	NA

Most of the entries in *ratings* are *NA* because each reviewer has only seen a small proportion of the movies. If we could impute those missing entries, we could estimate which movies a reviewer would rate highly and recommend them. Alternatively, if we could identify that movies A and B are similar, we could recommend movie B if a reviewer enjoyed movie A. Here we will use SVD to determine similar movies based on the small portion of available reviews in *ratings*.

Let

$$I = \{ij : ratings_{ij} \neq NA\}$$

be the entries of *ratings* which have been recorded. We will impute the missing entries using the iterative method discussed in class and summarized here. Let $X^{(1)}, X^{(2)}, \ldots$ be a sequence of matrices with the same dimensions as *ratings* and $\hat{X}_k^{(1)}, \hat{X}_k^{(2)}, \ldots$ be their rank k approximations respectively computed by setting the $(k+1)$th to last singular values to zero. Finally, $\ell^{(1)}, \ell^{(2)}, \ldots$ is a sequence of errors

$$\ell^{(j)} = \sum_{ij \in I} |ratings_{ij} - (\hat{X}_k^{(j)})_{ij}|^2$$

between the observed ratings and the corresponding entries in $\hat{X}_k^{(j)}$. We proceed as follows.

i) Set

$$X_{ij}^{(1)} = \begin{cases} ratings_{ij} & ij \in I \\ \bar{x}_j & ij \neq I \end{cases}$$

where $\bar{x}_j$ is the average of the observed entries in the jth column of $\mathbf{X}$. Let $\hat{X}_k^{(1)}$ denote the rank k approximation to $X^{(1)}$. Compute $\ell^{(1)}$.

ii) Given $\hat{X}_k^{(j-1)}$ set

$$X_{ij}^{(j)} = \begin{cases} ratings_{ij} & ij \in I \\ (\hat{X}_k^{(j-1)})_{ij} & ij \neq I \end{cases}.$$

Compute $\hat{X}_k^{(j)}$ and $\ell^{(j)}$.

iii) Repeat step (ii) until $\ell^{(j)}$ converges.

Assuming that this algorithm converges at step j^*, we can then use $X^{(j^*+1)}$ as our matrix of imputed ratings.

a) What proportion of the reviews are missing?

```
if (!exists("ratings")) {
  if (exists("rating")) ratings <- rating else if (exists("rating.matrix")) ratings <- rating.matrix els
}
prop_missing <- mean(is.na(ratings))
prop_missing
```

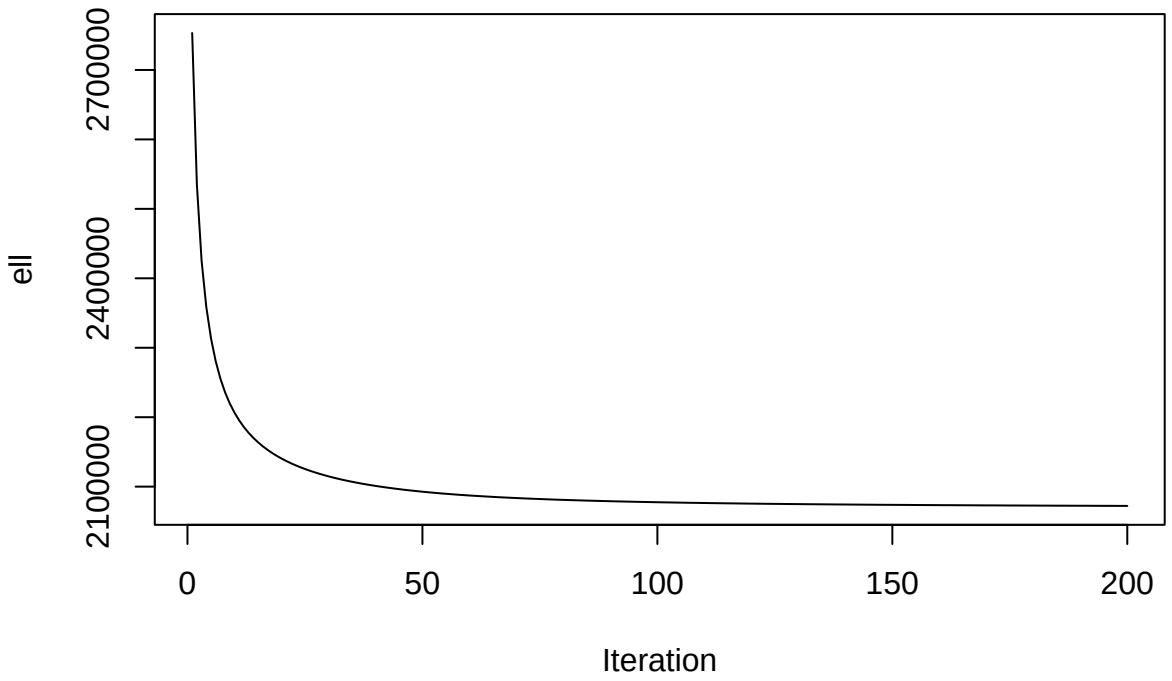
```
## [1] 0.7798536
```

About 78.0

b) Run the algorithm for 200 steps with $k = 10$. Plot the loss $\ell^{(j)}$ over those 200 iterations and the first five rows and three columns of the approximate ratings. Given the size of the *ratings* matrix, do not use the standard *svd()* command. Instead, use the approximate method *out <-irlba(A, nv = k, nu = k)* when you want to compute the first k singular vectors of A. The left singular vectors, singular values, and right singular values may be found in *out$u*, *out$d*, and *out$v* respectively.

```
k <- 10
R <- ratings
I_obs <- !is.na(R)
cm <- colMeans(R, na.rm = TRUE)
```

```
X <- R
for (j in 1:ncol(X)) X[is.na(X[, j]), j] <- cm[j]
loss <- numeric(200)
hatX <- matrix(0, nrow = nrow(X), ncol = ncol(X))
for (t in 1:200) {
  out <- irlba(X, nv = k, nu = k)
  hatX <- out$u %*% diag(out$d, nrow = k, ncol = k) %*% t(out$v)
  diff <- R[I_obs] - hatX[I_obs]
  loss[t] <- sum(diff^2)
  X[!I_obs] <- hatX[!I_obs]
}
plot(seq_len(200), loss, type = "l", xlab = "Iteration", ylab = expression(ell))
```




```
knitr::kable(hatX[1:5, 1:3], "simple")
```

2.830221	2.051025	3.682770
3.882176	2.404757	4.581611
3.400542	2.682456	3.101713
3.551734	3.767225	2.655679
5.085316	4.534394	3.160932

The loss dropped quickly early, then flattened and stabilized by roughly iterations 150–200. The curve shows steady convergence with diminishing returns after the first 50 iterations.

c) Compute the right singular vectors of the matrix $X^{(201)}$ and save them to a matrix V. Each of the columns of $V^T = [\vec{\xi}_1 | \dots | \xi_d]$ corresponds to one of the movies in the data set where $\vec{\xi}_j \in \mathbb{R}^k$. The cosine similarity between movies j and k is

$$sim(j,k) = \frac{\vec{\xi}_j^T \vec{\xi}_k}{\|\vec{\xi}_j\| \cdot \|\vec{\xi}_k\|}.$$

Large positive (negative) values of $sim(j, k)$ indicate that movies j and k are very similar (dissimilar). Based on this convention, which pair of movies are most similar? Which are most dissimilar?

```
out_final <- irlba(X, nv = k, nu = k)
V <- out_final$v
Xi <- V
rn <- colnames(ratings)
if (is.null(rn)) rn <- as.character(seq_len(ncol(ratings)))
rownames(Xi) <- rn
row_norms <- sqrt(rowSums(Xi^2))
Z <- Xi / row_norms
sim_mat <- Z %*% t(Z)
rownames(sim_mat) <- rn
colnames(sim_mat) <- rn
ut <- which(upper.tri(sim_mat), arr.ind = TRUE)
best_idx <- ut[which.max(sim_mat[ut]), ]
worst_idx <- ut[which.min(sim_mat[ut]), ]
most_similar <- c(rownames(sim_mat)[best_idx[1]], colnames(sim_mat)[best_idx[2]])
most_dissimilar <- c(rownames(sim_mat)[worst_idx[1]], colnames(sim_mat)[worst_idx[2]])
most_similar
```

```
## [1] "Lord of the Rings: The Two Towers, The (2002)"
## [2] "Lord of the Rings: The Return of the King, The (2003)"
```

```
most_dissimilar
```

```
## [1] "Sneakers (1992)"                "Blair Witch Project, The (1999)"
```

Most similar: Lord of the Rings: The Two Towers, The (2002) and Lord of the Rings: The Return of the King, The (2003). This makes sense. Same franchise, similar audience patterns.
Most dissimilar: Sneakers (1992) and Blair Witch Project, The (1999). Genres and viewing patterns differ, so the model separates them.

d) Suppose a reviewer enjoyed Toy Story (column 1 of ratings). Using the methodology from part c, recommend 5 movies that they may also like.

```
toy_col <- 1
sim_vec <- sim_mat[toy_col, ]
sim_vec[toy_col] <- -Inf
rec_idx <- order(sim_vec, decreasing = TRUE)[1:5]
recommended <- names(sim_vec)[rec_idx]
recommended
```

```
## [1] "Toy Story 2 (1999)"          "Bug's Life, A (1998)"
## [3] "Finding Nemo (2003)"         "Monsters, Inc. (2001)"
## [5] "Beauty and the Beast (1991)"
```

Toy Story 2 (1999)
A Bug's Life (1998)
Finding Nemo (2003)
Monsters, Inc. (2001)
Beauty and the Beast (1991)